SecureDT - Project Final Report

Intelligent and Secure Task Scheduling for Digital Twin-Enabled Edge-Cloud Networks

SecureDT is a research prototype for intelligent and secure task scheduling in digital twin-enabled edge-cloud networks. This report summarises the delivered system, its engineering, and the evaluation evidence.

1. Delivered System

- Django 5.2 server exposing pages and a token-authenticated REST API (securedt project).
- MEC agent (Python/PyInstaller) with pairing-code registration, heartbeat lifecycle, task execution and auto-reconnect.
- Core engines: digital twin, genetic algorithm scheduler, risk engine, attack simulator, security monitor, telemetry, AI (Random Forest + optional Groq LLM).
- Cloud backend: Google Drive integration proxied through the Django cloud app.
- Data pipeline: dataset registry, generation, validation, quality, normalization, big-data streaming (up to 10M rows).

2. Technical Stack

	Technology
	Django 5.2, Django REST Framework, CORS, channels (optional)
	scikit-learn Random Forest (RandomizedSearchCV), shap/xgboos
	Google Drive API via service account
	psutil, requests, PyInstaller
	Vanilla ES6 + Pico.css, Chart.js, Three.js

3. Scheduling & Optimization

The proposed scheduler is a security- and privacy-aware genetic algorithm minimizing latency, energy, privacy loss, security risk and a quadratic deadline penalty. OFFLINE MECs are never assignable. Baselines: random, greedy, privacy_aware.

4. Security Engineering

- CSRF protection on state-changing calls; session + DRF token authentication on /api.
- Django security middleware; signed sessions; secrets kept in .env (never committed).
- Upload validation: CSV only, 8 MB and 100,000-row caps.
- MEC auth with rotating bearer tokens (max age 3600 s).
- Structured audit log (datasets/audit_log.csv).
- All attack simulations are virtual; nothing external is touched.

5. Evaluation Method

Four schedulers are compared on shared seeded workloads via experiment campaigns, measuring success rate, average latency, energy, privacy exposure and security risk. Results append to datasets/experiment_results_2000.csv and are visualised in /analytics.

6. Reproducibility

- Fixed seed config.DATASET_SEED = 42; deterministic GA RNG.
- python scripts/generate_dataset.py --tasks 10000, python scripts/train_ai.py.
- python tests/run_all.py runs the unit + integration suite.

7. Limitations

- Single Django instance; not distributed.
- Energy is software-estimated; hardware meters needed for precise figures.
- Attack simulation is digital-twin-only.
- Rate limiting and CSP are configured as constants/report cards; a production deployment should add enforced limiter middleware.